Legacy Banking Modernisation Platform

Round 2 edition — proving the rewrite behaves exactly like the original

Team StudyEdge · Odisha University of Technology and Research, Bhubaneswar
CII Eastern Region Hackathon · Theme: Banking and Finance Technology
Submission: 13 August 2026 · Finals: ICT East, Kolkata, 24 September 2026

Supersedes the round-1 report (4 August 2026). New in this edition: target users and stakeholders, technical details, market potential, and the business-plan sections (investments, returns, timelines) required by the CII template. Claims sourced in `Research Work/01–08`; team-authored estimates are marked as assumptions where they occur.

The one-line thesis

Legacy modernisation fails on verification, not translation. We make behavioural equivalence something a bank can actually prove — and then check that answer against the regulator.

Part 1 — The problem

The situation

Indian banks, insurers and public-sector institutions run their core operations on COBOL programs written in the 1980s. 92 of the world's top 100 banks run core operations on IBM mainframes; more than 40% of banking systems still run COBOL. Everyone involved knows this is a liability. Almost nobody has managed to leave.

Take one program — the engine that calculates interest on savings accounts. Four thousand lines. Written in 1987. Modified roughly two hundred times since. Every modification was a response to something real: a regulatory change, a leap-year bug, a rounding complaint, a fix for accounts opened on the 31st. None of it was documented. The engineers who made those changes have retired.

The misdiagnosis

The common assumption is that modernisation is hard because translating COBOL into Java is hard. It isn't. Commercial transpilers have existed for decades; AWS Transform for mainframe went generally available in May 2025; refactoring platforms are a mature product category. Translation is the solved half.

It was never the half anyone was stuck on.

The real problem

Where is the document that states what the program is supposed to do? There isn't one. It was never written. **The code is the specification.** The only ways to learn a business rule are to read four thousand undocumented lines, to run the program and observe it, or to ask someone who retired in 2009.

Why nobody risks it

The rewrite compiles. It passes the tests the team wrote. Should the bank deploy it? Nobody can answer, because nobody knows what the old program does in full. And the tolerance for error is zero:

Failure mode	Consequence
Rounding drift	Silent, compounding, discovered at audit
Date boundary handling	Loans mature on the wrong day
Threshold logic	Wrong rate applied to an entire tier

A discrepancy of one paisa across millions of accounts is a reportable regulatory event. The cost of getting this wrong is not hypothetical:

- **TSB, United Kingdom, 2018.** A core migration failed at cutover. The FCA and PRA fined TSB £48.65m for operational risk and governance failings; £32.7m went in customer redress; total direct cost was around £400m. The CEO resigned and the former CIO was personally fined £81,620. The regulator's finding was about *risk management and assurance* — not about programming.
- **HDFC Bank, India, 2020.** After repeated outages, RBI halted every Digital 2.0 launch and froze new credit-card issuance. The card freeze lifted only in August 2021; the digital freeze ran into 2022. In India, an IT failure in a bank is a supervisory event that freezes the product roadmap.

So the institution keeps running software it cannot safely change — which is the correct decision under its own risk framework. Multiply it across thousands of institutions and the backlog is measured in decades.

The bottleneck is verification, not code generation.

Part 2 — Target users and stakeholders

Stakeholder	Role	Pain the platform addresses
Bank CIO / head of core banking	Owns the migration decision and the cutover date	Cannot sign off a replacement nobody can prove behaves identically
Risk, compliance and internal audit	Owns the board-approved Change Management Policy required by RBI's IT Governance Master Direction (2023)	No auditable artefact showing old and new agree, case by case
Modernisation vendors and system integrators	Cognizant, TCS, Infosys and in-house delivery teams	Translation is commoditised; assurance is what stretches the programme
The supervisor	RBI, and the bank's own board	An IT failure is a supervisory event, not an engineering incident

Nobody on this list is short of a translator. Everybody on this list is short of proof. The buyer is the person who has to sign the cutover memo — and today has nothing to attach to it.

(Roles are sourced; the pain statements are the team's inference and are recorded as such in `08 §1`.)

Part 3 — The solution

Why the obvious approach fails

Point a language model at the COBOL and ask it to explain the business rules, and the output looks excellent — clean, confident, well organised. And some percentage of it is wrong. A confidently stated wrong rule is more dangerous than no rule at all, because an engineer will build on it. Extraction cannot be the product. **Verification has to be the product.**

The pipeline

A deterministic pipeline with the AI on a short leash — the LLM is called at two bounded points inside a fixed sequence, never as a free agent:

1. **Intake gate.** Programs that read clocks, databases or CICS screens are detected and *refused with a reason* — differential testing is meaningless on non-deterministic programs, so the tool says no rather than mishandling them.
2. **Parse and slice.** ProLeap builds the AST and semantic graph; program slicing on business-concept variables produces bounded units.

3. **Extract rules** — one bounded LLM call per slice, with line citations. The slice defines the line set, so the model cannot invent a citation.
4. **Rule-directed input generation.** The extracted rules say where the boundaries are: ₹99,999 / ₹1,00,000 / ₹1,00,001 (RBI's uniform-rate threshold), accruals landing on exact ₹0.50 (the nearest-rupee rounding midpoint), 28/29 February, month and quarter ends, zero, negative and maximum values.
5. **Execute and compare — three ways.** The same input goes through the legacy COBOL (compiled via GnuCOBOL — the oracle), the generated Java (untrusted by design), and a clause-cited model of the RBI Master Direction. Outputs are diffed exactly.

The key idea: the legacy system is its own oracle

Testing normally requires knowing the correct answer in advance. Here that requirement disappears, because the program being replaced already defines the correct answer — by running. Every mismatch becomes a concrete, reproducible finding rather than a confidence score:

> Given a balance of 9,999.995 on a leap-year accrual, the legacy program returns 1042.50 and the reimplementation returns 1042.51.

The reciprocal loop

Extraction and verification strengthen each other. If the rules say balances below ₹1 lakh earn a uniform rate, the input generator now knows where the boundary is and tests either side of it. The comprehension layer makes the testing sharper; the testing layer catches the comprehension layer lying. Neither works alone.

Honesty as a feature

Every extracted rule ends in one of three verdicts:

- **Verified** — the corpus exercised it and all sides agreed on every case.
- **Unproven** — extracted but never exercised; flagged, not silently trusted.
- **Refuted** — the execution contradicted the stated rule; the extraction layer was caught being wrong.

Coverage is reported rather than assumed. The same honesty applies at intake: the tool tells you when it cannot help you.

The regulatory third reference

The RBI Master Direction (Interest Rate on Deposits), 2016 — updated 2024 — fixes the arithmetic in writing: daily product basis (clause 6(a)); a uniform rate up to ₹1 lakh (6(a)(1)–(2)); interest rounded to the nearest rupee for rupee deposits (4(f)), with 50 paise and above rounding up. Checking both programs

against the regulator's own arithmetic means the platform can catch the case where the legacy itself is non-compliant — something a pure old-vs-new diff can never see.

Part 4 — Technical details

Layer	Component	Why it is there
Legacy execution	**GnuCOBOL** (cobc/libcob, GPL/LGPL)	19 COBOL dialects; `-std=ibm-strict` against Enterprise COBOL 6.3; compiles to native binaries that can be run, timed and instrumented. Passes 9,700 of 9,748 NIST CCVS85 tests — while the project itself claims no formal conformance, a caveat we state ourselves.
Parsing	**ProLeap COBOL parser** (ANTLR4, MIT)	AST plus abstract semantic graph with data and control flow; handles COPY/REPLACE; extracts `EXEC SQL` / `EXEC CICS` as text, which is what makes the intake gate's refusals detectable rather than accidental.
Extraction	Program slicing + bounded LLM calls	A slice is a set of source lines — exactly the traceability promised. One call per slice bounds context and makes line citations mechanical.
Coverage	**gcov** (via GnuCOBOL's C output)	Exact statement execution counts; fallback is a paragraph-level trace via `-fgen-c-line-directives`. This is how the verified/unproven verdict is computed.
Comparison	Differential runner + **delta debugging**	Same input, three outputs, exact diff; delta debugging shrinks a pile of failing inputs to one minimal counterexample.
Corpora	**AWS CardDemo** (Apache 2.0), **NIST CCVS85** (public domain), own RBI-modelled fixture	Third-party COBOL avoids the circularity of testing our tool on code we wrote. CardDemo's interest calculator reads VSAM files and the clock, so the intake gate correctly refuses it — which is itself a demo.

Integration: CLI plus a JSON evidence bundle. The methodology is compiler-agnostic — swap GnuCOBOL for IBM Enterprise COBOL inside the bank and the harness is unchanged. No mainframe access is required for the prototype.

Why these technologies: everything is free and citable, so the prototype runs end-to-end on a laptop. Deterministic orchestration with bounded LLM calls matches agentic accuracy at up to 3.5× lower token cost, with lower run-to-run variance (arXiv 2605.09894) — and a harness that compares programs for exact equality cannot itself be non-deterministic.

Part 5 — Market potential

The spend is already committed. BCG estimates Indian lenders must invest roughly **$1bn over 5–10 years** to upgrade legacy core banking systems. Indian banks spend up to 5% of revenue on IT against 7–9% for global peers — a gap that compounds into more accumulated undocumented logic. RBI's ombudsman recorded over 40,000 mobile and internet banking complaints across FY22–23; the supervisor is already acting on IT fragility.

Every rupee of that spend is gated by one step. What stalls approved modernisation budgets is the assurance step between "the new code exists" and "we are willing to run it on real money". That step has no product category today — it is absorbed as consulting hours and schedule risk.

The category is forming, and it is forming around generation. AWS Transform for mainframe (GA May 2025; ADP cut rule-extraction time by 80% and manual effort by over 90%), IBM watsonx Code Assistant for Z (whose validation assistant exists because IBM's own papers say translated code cannot be trusted unchecked), and Mechanical Orchard's Imogen (a generate-validate loop) all *generate first* and validate as a feature. None of them sells the proof as the product.

(The gating claim is positioning — the team's synthesis from the competitive landscape — not a market statistic; recorded as such in `08 §2`.)

Part 6 — What makes it different

Stated with the verification pass (`07`) in view, so nothing here overclaims:

1. **The intake gate refuses what it cannot verify soundly** — with a reason, on detection. The honesty argument applied before any analysis begins.
2. **The reciprocal loop with per-rule verdicts.** Shipping tools treat extraction and verification as separate phases; the academic literature splits the same way. Closing the loop — rules direct the inputs, execution then judges the rules, coverage is reported per business rule as verified / unproven / **refuted** — is the part that is genuinely thin in both products and papers.
3. **The regulatory third reference.** Diffing old against new proves equivalence; diffing both against the RBI's written arithmetic can prove the legacy itself wrong. No adjacent product does this.

The differential mechanism itself is prior art — IBM, AWS and Mechanical Orchard ship it, and Locksmith (arXiv 2607.28271) builds the same spine off-mainframe. That convergence validates the problem; the three items above are the contribution.

Part 7 — Business plan

Investments — what it takes

Item	Detail
Licence cost	**Nil.** GnuCOBOL (GPL/LGPL), ProLeap (MIT), gcov, NIST CCVS85 and AWS CardDemo are all open. No mainframe, no vendor contract.
Prototype	4 engineers × 12 weeks on commodity laptops *(effort-based estimate)*
Variable cost	LLM inference only — one bounded call per slice; deterministic orchestration at up to 3.5× lower token cost than agentic
Bank pilot	2 engineers plus bank SME time, one program family, ~6 months *(estimate)*

Returns — and what if I don't solve?

- **Unlocks:** the ~$1bn gated modernisation spend; ADP-comparable automation (−80% extraction time, −90% manual effort) on the extraction stage; review cost per rule drops from four thousand lines to three.
- **The counterfactual:** TSB 2018 — ~£400m, a resigned CEO and a personally fined CIO — for want of pre-cutover assurance. In India, HDFC 2020 shows the same failure freezing a roadmap for over a year.
- No specific ROI multiple is claimed; the counterfactual prices the downside at two orders of magnitude above the platform's cost.

Timelines — time to realise benefit

When	Deliverable
Weeks 1–2	Differential harness runs two programs on an input list and diffs — already a working demo
Weeks 3–4	Input generation from PIC clauses plus hand-written boundaries
Weeks 5–7	Parse, slice, extract rules with line citations
Weeks 8–10	**The reciprocal loop and per-rule coverage verdicts — the contribution**
Weeks 11–12	Delta debugging to minimal counterexamples; evidence bundle
Months 4–9	Bank pilot on one program family, inside the bank's own compiler environment

(Stage contents and risks researched in `04 §7`; the calendar mapping is the team's own sizing.)

Part 8 — Scope and feasibility

In scope: COBOL compiled and executed via GnuCOBOL; retail banking interest accrual; pure computation — input in, output out; third-party COBOL including AWS's own CardDemo sample.

Out of scope, detected rather than ignored: whole-system migration; CICS screens and JCL orchestration; database-coupled programs; anything non-deterministic (clocks, randomness, external state). Differential testing compares outputs for equality, so non-determinism makes the verdict meaningless — the tool refuses these at intake, with a reason.

Known limitation: input-generation quality caps everything. Naive random inputs will not find a rounding bug. That is precisely why the reciprocal loop is where the build effort goes.

Part 9 — Outcome

The institution keeps four artifacts:

1. **A specification** — readable, traceable documentation for a system that never had any.
2. **A regression suite** — thousands of input–output pairs, retained permanently.
3. **An equivalence claim** — evidence-backed, with coverage stated honestly, and the artifact someone can actually sign.
4. **Compliance findings** — the legacy checked against RBI's written arithmetic, not only against itself.

The first two hold their value even if the migration is never approved.

The pitch in one sentence

The industry's most serious players converged on the same bottleneck we did — verification — and every one of them sells generation with validation bolted on. We build the proof as the product, report the coverage we do not have, and check both programs against the regulator.

Sources: `Research Work/06 - Sources.md` *and* `Research Work/citations and research paper/` *(references.bib, annotated bibliographies, verified quotes). Evidence and assumptions for the business-plan sections:* `Research Work/08 - Business Plan Research.md`.